

Команда 9: Злата Леськів, Рінг Анна, Фалес Олександр, Остап Сухомлин

Тема: Конволюційні Коди

Ментор: Марина Огінська

Звіт по виконаній Роботі

Даний проєкт реалізує комбінований підхід до обробки інформації: стиснення (для зменшення обсягу) та конволюційне кодування (для захисту від помилок у каналі з шумом). Нижче наведено детальний аналіз кожного компонента системи.

1. Кодер (coder.py)

Модуль реалізує конволюційний кодер зі швидкістю (rate) 1/3. Це означає, що на кожен вхідний біт генерується три вихідних.

Як він працює:

- Стани: Кодер використовує 4 можливих стани (2^2), що базуються на внутрішній пам'яті (реєстрі зсуву).
- Таблиця переходів (build_table): При ініціалізації програма заздалегідь розраховує всі можливі переходи. Для кожної пари (поточний стан, вхідний біт) визначається наступний стан та три вихідних біти.
- Алгоритм XOR: Вихідні біти розраховуються за допомогою операції XOR між вхідним бітом та значеннями в реєстрах. Це створює "залежність" кожного вихідного біта від попередніх, що і дозволяє відновлювати дані при пошкодженнях.

2. Декодер (decoder.py)

Цей модуль виконує зворотну операцію за допомогою алгоритму Вітербі — найефективнішого методу декодування конволюційних кодів.

Ключові етапи:

- Метрика Евкліда: Функція euclidean оцінює "відстань" між отриманим зашумленим сигналом та ідеальним очікуваним бітом.
- Побудова решітки (Trellis): Програма ітерується крізь отримані біти, обчислюючи cost (вартість) переходу в кожен із 4-х станів та зберігаючи найімовірніший шлях.
- Зворотний шлях (Backtracking): Алгоритм обирає стан з найменшою метрикою і проходить назад до початку, відновлюючи біти.

3. Симуляція (simulation.py)

Модуль дозволяє перевірити, наскільки ефективно код захищає дані від реального шуму.

Процес моделювання:

1. AWGN (Адитивний білий гаусів шум): Функція `add_awgn` імітує фізичний канал, додаючи випадковий шум до сигналу залежно від параметра SNR.
2. BER (Bit Error Rate): Розраховується частота помилок.
3. Візуалізація: Результати виводяться на графік порівняння "Coded vs Uncoded".

4. Головний модуль (main.py)

Це "центр керування", який об'єднує всі файли та додає функціонал стиснення за алгоритмом Хаффмана.

Інструкція з використання

Всі модулі запускаються через термінал. Переконайтеся, що ви знаходитесь у директорії з проектом.

Запуск головного модуля (main.py)

Модуль використовує бібліотеку `argparse` для обробки параметрів командного рядка.

1. Порівняння ефективності (Хаффман vs Конволюційне):
Ця команда проаналізує вхідний файл, покаже час кодування та розмір даних після стиснення та після захисного кодування.
`python main.py compare <шлях_до_файлу>`
Наприклад:
`python main.py compare input.txt`
2. Закодувати файл конволюційним кодом:
Створює бінарний файл із захищеними даними.
`python main.py encode_conv <вхідний_файл> -o <вихідний_файл>`
Наприклад:
`python main.py encode_conv text.txt -o encoded.bin`
3. Декодувати файл (Алгоритм Вітербі):
Відновлює оригінальні дані з бінарного файлу.
`python main.py decode_conv <закодований_файл> -o <відновлений_файл>`
Наприклад:
`python main.py decode_conv encoded.bin -o recovered.txt`

Запуск симуляції каналу AWGN (simulation.py)

Для запуску моделювання завадостійкості та побудови графіків BER достатньо виконати скрипт без додаткових параметрів:

```
python simulation.py
```

Що станеться після запуску:

- Програма виконає серію тестів для різних значень E_b/N_0 (від -2 до 6 дБ).
- У консолі ви побачите поточний прогрес та значення BER для кодованого та некованого сигналів.
- По завершенню відкриється вікно з графіком (використовується matplotlib), де червона лінія показує помилки без коду, а синя — з використанням розробленого кодера/декодера.